

Projekt Eyes of the Abyss

Zespół programistyczny: Kirill Siarchenia, Nathaniel Kusal

Czas realizacji: od 10.04.2026 od 08.06.2026

1. Ogólny opis projektu

1.1. Cel projektu

Celem projektu było stworzenie przeglądarkowej gry zręcznościowo-logicznej w klimacie mrocznego fantasy o nazwie "Eyes of the Abyss". Gra łączy w sobie dwa główne tryby rozgrywki: dwuwymiarową eksplorację labiryntu z widokiem z góry (renderowaną na elemencie HTML5 Canvas) oraz interaktywne sekcje point-and-click realizowane wewnątrz dedykowanych pokoi z zagadkami. Projekt został zrealizowany przy użyciu czystych technologii webowych (HTML5, CSS3, JavaScript ES6) bez użycia zewnętrznych silników i frameworków, co pozwoliło zachować pełną kontrolę nad wydajnością oraz architekturą aplikacji.

1.2. Charakterystyka gry

Gracz wciela się w postać, która schodzi do tajemniczej jaskini w poszukiwaniu zaginionego brata. Rozgrywka opiera się na przechodzeniu kolejnych poziomów o rosnącym stopniu trudności. Aby ukończyć dany etap, gracz musi odnaleźć zardzewiały klucz i dotrzeć do drzwi wyjściowych. Klucze są ukryte w pokojach z zagadkami logicznymi, do których wejście wymaga uprzedniego zbadania labiryntu.

Eksplorację utrudniają patrolujący korytarze przeciwnicy, reagujący na bodźce wzrokowe oraz słuchowe (dźwięki kroków i biegu gracza). Gra zawiera zintegrowany system przerywników filmowych (cutszenek) przedstawiających fabułę, pełne udźwiękowienie z dynamicznym systemem głośności, a także pełne wsparcie dla trzech wersji językowych: polskiej, angielskiej i rosyjskiej.

2. Wymagania funkcjonalne

2.1. Sterowanie postacią

- Ruch postaci: Przemieszczanie po siatce korytarzy labiryntu za pomocą klawiszy W, A, S, D / Strzałek.
- Bieg: Przytrzymanie klawisza Shift przyspiesza ruch postaci.
- Interfejs i pauza: Klawisz Escape lub P odpowiada za wywołanie menu pauzy, zamknięcie ekwipunku, cofnięcie akcji lub zamknięcie okna dialogowego.
- Płynność ruchu: Postać porusza się ze stałą prędkością, a ruch między polami (kafli) jest płynnie interpolowany czasowo (dt). Animacja sprite'a postaci zmienia kierunek patrzenia (góra, dół, lewo, prawo) oraz fazę ruchu w zależności od wektora przemieszczenia.

2.2. Mechanika labiryntu

- Siatka kafelków (Grid-based): Poziomy generowane są na bazie dwuwymiarowych tablic, gdzie każdy kafelek ma rozmiar 90x90 pikseli.
- Kolizje: System automatycznie blokuje ruch gracza oraz przeciwników w przypadku próby wejścia na kafelek oznaczony jako ściana (1) lub krawędź mapy lub inna encja.
- Interaktywne przejścia: Kafelki o wartościach xy(x - numer poziomy, y - numer pokoju) lub wyższych oznaczają wejścia do pokoi logicznych. Wejście do zablokowanego pokoju wywołuje komunikat o blokadzie i odtwarza dźwięk zamkniętych drzwi.
- Kamera: Obraz z kamery jest centrowany na postaci gracza. Kamera wykorzystuje system strefy martwej (dead zone) – zaczyna się przesuwać dopiero wtedy, gdy gracz zbliży się do krawędzi aktualnego obszaru widzenia.

2.3. System przedmiotów

- Zbieranie przedmiotów: Gracz może zbierać przedmioty leżące bezpośrednio na posadzce labiryntu oraz przedmioty ukryte w pokojach.
- Typy przedmiotów:
 - Materiały zużywalne
 - Przedmioty użytkowe/zagadkowe
- Zarządzanie ekwipunkiem: Przedmioty trafiają do modalnego okna ekwipunku, które można wywołać wewnątrz pokoi. Każdy przedmiot posiada opcję "Zbadaj" (wyświetla opis fabularny) oraz "Użyj".
- Używanie przedmiotów: Wybranie przedmiotu do użycia ukrywa okno ekwipunku, blokuje standardowy wskaźnik systemowy i aktywuje niestandardowy kursor myszy przedstawiający ikonę wybranego przedmiotu.

2.4. Zagadki logiczne

Pokoje point-and-click składają się z trzech widoków (lewy, środkowy, prawy), między którymi gracz przełącza się za pomocą strzałek ekranowych.

2.5. System przeciwników

- Detekcja gracza (Wzrok i Słuch): Przeciwnicy wykrywają gracza na podstawie prostej linii wzroku (brak ścian na drodze wektora ruchu) lub na podstawie hałasu (gdy gracz biegnie lub idzie w odległości mniejszej niż zdefiniowany zasięg słuchu).
- Maszyna stanów sztucznej inteligencji (AI):
 - Patrol - przemieszczanie się po z góry określonej pętli punktów patrolowych.
 - Chase (Pościg) - dynamiczne wyznaczanie trasy bezpośrednio do pozycji gracza.
 - Searching (Poszukiwanie) - aktywowane w momencie zgubienia gracza.

Składa się z faz: reachLocation (dotarcie do miejsca ostatniego kontaktu), inertia (ruch bezwładny w kierunku ucieczki gracza), lookAround (rozejrzenie się w dostępne kierunki). Po zakończeniu poszukiwania wróg wraca do patrolowania.

- Algorytm przeszukiwania drogi: Wykorzystanie algorytmu A* (A-Star) opartego na metryce Manhattan do wyznaczania najkrótszej bezkolizyjnej ścieżki w czasie rzeczywistym.
- Mechanika obrażeń: Kontakt fizyczny z przeciwnikiem skutkuje odjęciem 1 punktu życia, wywołaniem animacji uderzenia (slash), odtworzeniem efektów dźwiękowych oraz odrzuceniem gracza (knockback) o maksymalnie dwa kafelki wstecz. Gracz otrzymuje czasową niewrażliwość (1 sekunda) sygnalizowaną miganiem postaci.

2.6. System poziomów

Gra oferuje cztery zdefiniowane poziomy o rosnącej trudności:

1. Poziom 1: Prosty układ korytarzy, dwóch przeciwników, podstawowa zagadka ze skrzynią.
2. Poziom 2: Większa mapa, trzech przeciwników, wieloetapowa zagadka z kołami zębatymi i dźwignią.
3. Poziom 3: Rozbudowany labirynt, czterech przeciwników, złożona zagadka z posągami.
4. Poziom 4 (Epilog): Specjalna mapa narracyjna.

2.7. Interfejs użytkownika (UI)

- Menu Główne: Zawiera logo gry, przyciski nawigacyjne ("GRAJ", "SAMOUCZEK", "USTAWIENIA") oraz tło graficzne zmieniające się na wersję alternatywną po jednorazowym ukończeniu gry.
- Wybór trudności: Menu pozwalające wybrać poziom "ŁATWY" (brak ograniczenia widoczności) lub "ZAMIERZONY" (aktywacja dynamicznej mgły wojny wokół gracza).
- Pasek zdrowia (HUD): Wizualna prezentacja życia za pomocą serc, które animują się (efekt utraty/leczenia) i aktualizują stan na bieżąco.
- System wiadomości: Okno dialogowe z płynnym wyświetlaniem tekstu znak po znaku (efekt maszyny do pisania), zintegrowane z losowym odtwarzaniem dźwięków uderzeń klawiszy. System zabezpieczony jest przed zjawiskiem "click-through" (kliknięcie zamykające dialog nie przenosi się na elementy pod nim).
- Ekran końcowe: Ekran śmierci ("Zginąłeś"), ekran zakończenia oraz sekcja twórców (napisy końcowe).

2.8. Funkcje dodatkowe

- Dźwięki: Osobne efekty dla kroków gracza, kroków przeciwnika (o głośności skalowanej dystansem), otwierania i blokowania drzwi, uderzeń, mechanizmów pokojowych oraz pisania na maszynie.
- System muzyki ambientowej: Odtwarzanie pętli ambientowych dostosowanych do stanu gry. Wyciszenie i przytłumienie (muffle effect) muzyki podczas pauzy.
- Wielojęzyczność (Lokalizacja): Dynamiczne tłumaczenie wszystkich tekstów menu, dialogów, przerywników i interakcji na języki polski, angielski oraz rosyjski.

3. Wymagania нефunkcjonalne

3.1. Wydajność

- Wydajność renderowania: Rendering oparty o natywny kontekst 2D Canvas zoptymalizowany pod kątem ograniczania operacji czyszczenia i przerysowywania. Wszystkie zasoby graficzne są ładowane jednorazowo przy inicjalizacji i przechowywane w pamięci podręcznej (itemAssets.cache).
- Optymalizacja audio: Ze względu na ograniczenia przeglądarek dotyczące jednoczesnego odtwarzania tych samych plików audio, system dźwięków klonuje węzły dźwiękowe (cloneNode()) w locie, co eliminuje opóźnienia i nakładanie się dźwięków.

3.2. Użyteczność

- Spójność graficzna: Wszystkie elementy wizualne zostały wykonane w jednolitej stylistyce mrocznego piksel-artu, co buduje gęstą atmosferę horroru.
- Dostępność językowa: Możliwość zmiany wersji językowej bezpośrednio z poziomu menu ustawień w dowolnym momencie rozgrywki.

3.3. Responsywność

- Aplikacja automatycznie dopasowuje wymiary pola Canvas do rozmiarów okna przeglądarki (resizeCanvas), przeliczając pozycje renderowania elementów w pokojach na podstawie bazowej rozdzielczości projektowej 1920x1080.

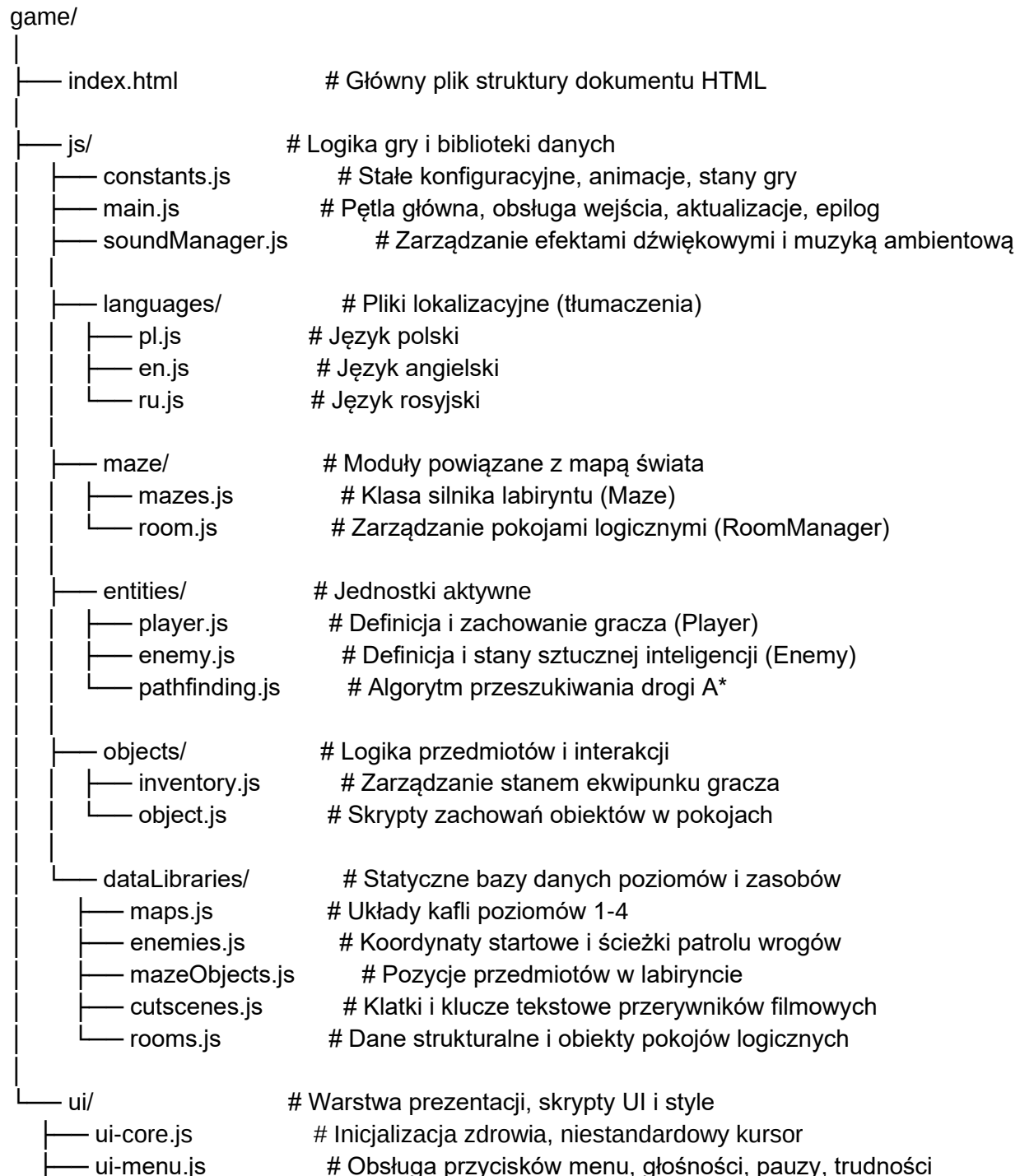
3.4. Bezpieczeństwo i stabilność

- Stan gry: Zapisywanie ustawień głośności muzyki, dźwięków, wybranego języka, poziomu trudności oraz faktu przejścia gry w bezpiecznym magazynie przeglądarki localStorage.
- Autopauza: Wyjście z trybu pełnoekranowego automatycznie wstrzymuje rozgrywkę i wyświetla dedykowany ekran blokady, zabezpieczając gracza przed utratą punktów życia w tle.

4. Architektura projektu

4.1. Struktura katalogów

Poniższy schemat przedstawia strukturę plików i folderów aplikacji:



— ui-inv.js	# Renderowanie slotów ekwipunku, akcje przedmiotów
— ui-msg.js	# Efekt maszyny do pisania, blokowanie click-through
— ui-cutsce.js	# Zarządzanie i odtwarzanie przerywników
— camera.js	# Klasa obsługująca ruch i strefy martwe kamery
— css/	# Arkusze stylów
— base.css	# Podstawowe reguły i filtry graficzne
— hud.css	# Interfejs zdrowia i paska szybkiego dostępu
— inventory.css	# Okno ekwipunku i menu kontekstowe przedmiotów
— menus.css	# Nakładki menu głównego, pauzy, śmierci i ustawień
— messages.css	# Wygląd okna dialogowego wiadomości
— navigation.css	# Przyciski nawigacji pokojowej (strzałki)
— cutscenes.css	# Stylizacja sekcji przerywników
— fonts/	# Czcionki
— assets/	# Elementy graficzne oraz dźwiękowe
— sounds/	# Pliki efektów audio oraz muzyki (.ogg)
— [pliki graficzne]	# Spritesheety, tła, ikony przedmiotów (.png)

4.2. Opis kluczowych modułów

- main.js (Pętla Gry): Odpowiada za rozruch aplikacji, synchronizację klatek animacji za pomocą requestAnimationFrame, zarządzanie przejściami między stanami (setGameState), detekcję interakcji, aktualizację logiki epilogu oraz renderowanie warstwy graficznej.
- player.js & enemy.js (Encje): Klasa Player przetwarza ruch bohatera, odrzut po obrażeniach oraz aktualizuje parametry życiowe. Klasa Enemy kontroluje ruch wrogów na bazie ich aktualnych stanów logicznych (patrol, pościg, szukanie), przetwarza linie wzroku i generuje dźwięki kroków.
- pathfinding.js (Wyszukiwanie Drogi): Implementacja klasy pomocniczej poszukiwania drogi A*. Oblicza optymalną ścieżkę jako sekwencję punktów siatki, pozwalając wrogom na omijanie ścian i inteligentne ściganie gracza.
- soundManager.js (System Audio): Wczytuje i grupuje efekty dźwiękowe oraz ścieżki tła. Kontroluje poziomy głośności, płynną modyfikację tempa odtwarzania (np. dla maszyny do pisania) oraz asynchroniczne wyciszanie muzyki podczas pauzy.
- room.js (RoomManager): Zarządza point-and-clickową przestrzenią pokoi. Inicjalizuje obiekty, interpretuje kliknięcia w obszary aktywne, zarządza zmianą widoków (lewo, środek, prawo) i wywołuje przypisane zachowania logiczne obiektów.

5. Dokumentacja kodu

5.1. Komentarze w kodzie

Kod źródłowy został opatrzony dwujęzycznymi komentarzami (język rosyjski i polski). Komentarze precyzyjnie wyjaśniają przeznaczenie poszczególnych funkcji interfejsu użytkownika, procesy czyszczenia timera pisania, modyfikację głośności, a także logikę detekcji specyficznych stanów obiektów wewnątrz pokojów zagadek.

5.2. Opis kluczowych funkcji i metod

`showMessage(text, position)` (w `ui-msg.js`)

Płynnie wyświetla tekst w oknie komunikatów za pomocą efektu maszyny do pisania.

- Parametry: `text` (tekst do wyświetlenia), `position` ('top' dla wyświetlania nad postacią lub 'bottom' dla standardowego HUD-u).
- Działanie: Czyści aktywne timery pisania, resetuje kontener tekstowy, po czym asynchronicznie dodaje znaki co 40 ms. Odtwarza modulowany dźwięk pisania typewriter.
- Zabezpieczenie przed kliknięciem przechodzącym: Tworzy tymczasowy przechwytywacz zdarzeń myszy na fazie przechwytywania (`true`), co blokuje natychmiastowe kliknięcie elementów interfejsu (np. przycisków pod spodem) przy szybkim zamykaniu okna dialogowego.

`findPath(startX, startY, targetX, targetY, maze)` (w `pathfinding.js`)

Oblicza najkrótszą bezkolizyjną ścieżkę z punktu startowego do celu w labiryncie.

- Działanie: Inicjuje listę otwartą i zamkniętą węzłów. Wyszukuje sąsiednie kafelki, pomijając ściany i krawędzie mapy. Oblicza współczynniki kosztu przejścia G (odległość od startu) oraz heurystykę H (odległość Manhattan do celu). Zwraca tablicę współrzędnych kolejnych kroków.

`update(dt)` (w `enemy.js`)

Główna metoda aktualizująca stan logiczny przeciwnika w każdej klatce gry.

- Działanie: Sprawdza sensorykę wroga (linia wzroku, odległość słuchowa od gracza). Przełącza stany sztucznej inteligencji. W przypadku stanu `chase` wyszukuje ścieżkę do aktualnej pozycji gracza. W stanie `searching` realizuje krok bezwładnościowy w kierunku ucieczki gracza, a następnie uruchamia sekwencję obracania się i rozglądania w korytarzu.

`applyKnockback(enemyGridX, enemyGridY, maze)` (w `player.js`)

Realizuje odrzut gracza po otrzymaniu obrażeń od wroga.

- Działanie: Oblicza wektor kierunkowy od przeciwnika do gracza. Próbuje przesunąć gracza o 2 kafelki w tym kierunku na siatce, sprawdzając w locie kolizje ze ścianami (`maze.isFreeCell`), aby zapobiec utknięciu postaci w elementach stałych.

`useItem(item)` (w `inventory.js`)

Obsługuje aktywację przedmiotu z ekwipunku gracza.

- Działanie: Jeśli przedmiot jest bezpośrednio zużywalny (np. jedzenie), wywołuje jego funkcję `action()` (modyfikacja zdrowia gracza) i usuwa go z ekwipunku. Jeśli przedmiot jest elementem zagadki (np. kamień, rubin), zamyka interfejs ekwipunku, zapisuje referencję w `UI.selectedItemForUse` i uruchamia niestandardowy kursor myszy z grafiką przedmiotu.

6. Zarządzanie projektem

6.1. Metodyka pracy

Projekt był prowadzony zgodnie z metodyką Kanban, wykorzystując narzędzia do wizualizacji przepływu zadań (np. Trello/GitHub Projects). Zadania podzielono na etapy:

1. Analiza: Opracowanie konceptu gry, zebranie wymagań technicznych i funkcjonalnych.
2. Projektowanie: Szkicowanie układów poziomów, przygotowanie bazy danych przedmiotów i struktury interakcji w pokojach.
3. Implementacja: Równoległe kodowanie mechanik silnika (fizyka, AI, rendering) oraz przygotowanie grafik i interfejsu UI.
4. Testowanie: Weryfikacja integralności kodu, detekcja kolizji, testowanie algorytmu przeszukiwania dróg oraz balansu rozgrywki.

6.2. Podział ról

- Kirill Siarchenia: Główny Programista JS / Architekt Rozgrywki (Lead Developer) – odpowiedzialny za stworzenie pętli głównej gry, silnika labiryntu, zachowań przeciwników i sztucznej inteligencji, systemu wyszukiwania dróg (A*), menedżera dźwięku, interakcji w pokojach, manualne testy funkcjonalne kolizji, poprawność działania zagadek logicznych, płynność przejść pomiędzy scenami oraz weryfikację poprawności tłumaczeń językowych.

point-and-click oraz integrację kodu lokalizacyjnego i UI. Współautor oprawy graficznej.

- Nathaniel Kusal: Projektant Graficzny Designer (Graphic Design) – odpowiedzialny za przygotowanie spójnej oprawy wizualnej w stylu pixel-art, stworzenie arkuszy animacji (spritesheetów) postaci gracza i przeciwników, tła menu, interaktywnych elementów pokoiów, ikon przedmiotów oraz grafik przerywników filmowych.

7. Możliwości dalszego rozwoju

Architektura gry "Eyes of the Abyss" została zaprojektowana z myślą o separacji danych konfiguracyjnych od właściwej logiki silnika. Dzięki temu aplikacja charakteryzuje się wysoką elastycznością, co stwarza dogodne warunki do wdrażania nowych funkcjonalności i rozbudowy zawartości przy minimalnym ryzyku regresji w istniejących modułach.

7.1. Skalowalność bazy danych przedmiotów i interakcji

Wszystkie przedmioty w grze są zdefiniowane deklaratywnie w obiekcie `objectsLibrary.js`, a ich zachowania i reakcje na otoczenie są powiązane z metodami w `objects.js`.

- Dodawanie nowych przedmiotów: Wprowadzenie nowego elementu (np. nowego rodzaju klucza, pochodni czy eliksiru) wymaga jedynie dodania rekordu do `ObjectsLibrary` ze zdefiniowanym indeksem graficznym (`spriteIndex`) lub ścieżką do grafiki oraz określeniem funkcji `action`.
- Nowe typy interakcji: Możliwe jest łatwe rozbudowanie metod w `objectLogic.js` bez konieczności ingerencji w system ekwipunku (`inventory.js`) czy pętlę renderowania.

7.2. Rozbudowa o nowe poziomy i konfiguracje przeciwników

Siatki kafelków poziomów oraz pozycje startowe obiektów są całkowicie oddzielone od kodu operacyjnego.

- Projektowanie map: Nowe poziomy mogą być dodawane jako dwuwymiarowe tablice liczbowe w pliku `maps.js` i rejestrowane w tablicy `allLevels` w pliku `main.js`.
- Dynamiczne rozmieszczanie przeciwników i przedmiotów: Pliki `enemies.js` oraz `mazeObjects.js` przechowują współrzędne powiązane z kluczami poziomów (np. `map1`, `map2`). Dodanie nowego przeciwnika sprowadza się do dopisania obiektu zawierającego pozycję startową oraz tablicę punktów patrolowych (`path`), które system wyszukiwania drogi (A^*) przetworzy automatycznie.

7.3. Rozszerzalność systemu lokalizacji językowych

System wielojęzyczności opiera się na dynamicznym mapowaniu struktur klucz-wartość z plików lokalizacyjnych znajdujących się w katalogu `js/languages/`.

- Dodawanie nowych wersji językowych: Wdrożenie nowego języka (np. niemieckiego) wymaga utworzenia pojedynczego pliku o analogicznej strukturze kluczy (np. `de.js` eksportującego obiekt `langDE`) oraz zarejestrowania go w obiekcie `allLanguages` w pliku `main.js`.
- Wsparcie interfejsu: Funkcja `UI.updateStaticTexts()` automatycznie mapuje nowe tłumaczenia na elementy drzewa DOM przy przełączeniu języka, eliminując potrzebę ręcznego sterowania etykietami tekstowymi w plikach HTML.

7.4. Modułowość pokoi logicznych (Point-and-Click)

Struktura pokoi zdefiniowana w `rooms.js` za pomocą obiektu `INITIAL ROOMS DATA` pozwala na łatwe kreowanie nowych przestrzeni i zagadek.

- Definiowanie widoków: Pokoje są opisane strukturą trójwidokową (`left`, `center`, `right`). Dodanie nowego pokoju wymaga jedynie zadeklarowania współrzędnych obiektów dla każdego widoku, wskazania ich tekstur oraz przypisania logiki interakcji, co pozwala na tworzenie bardziej złożonych zagadek wielopokojowych.

7.5. Rozbudowa warstwy audio

Menedżer dźwięków (`soundManager.js`) obsługuje mapowanie asynchroniczne zasobów dźwiękowych w pętli inicjalizacyjnej.

- Nowe efekty i utwory: Wprowadzenie dodatkowych odgłosów interakcji lub nowych motywów muzycznych (ambientów) wymaga wyłącznie dodania nowej pary klucz-wartość wewnątrz obiektów `soundFiles` lub `ambientFiles`. System automatycznie obsługuje klonowanie węzłów i nakładanie efektu przytłumienia dźwięku podczas pauzy.